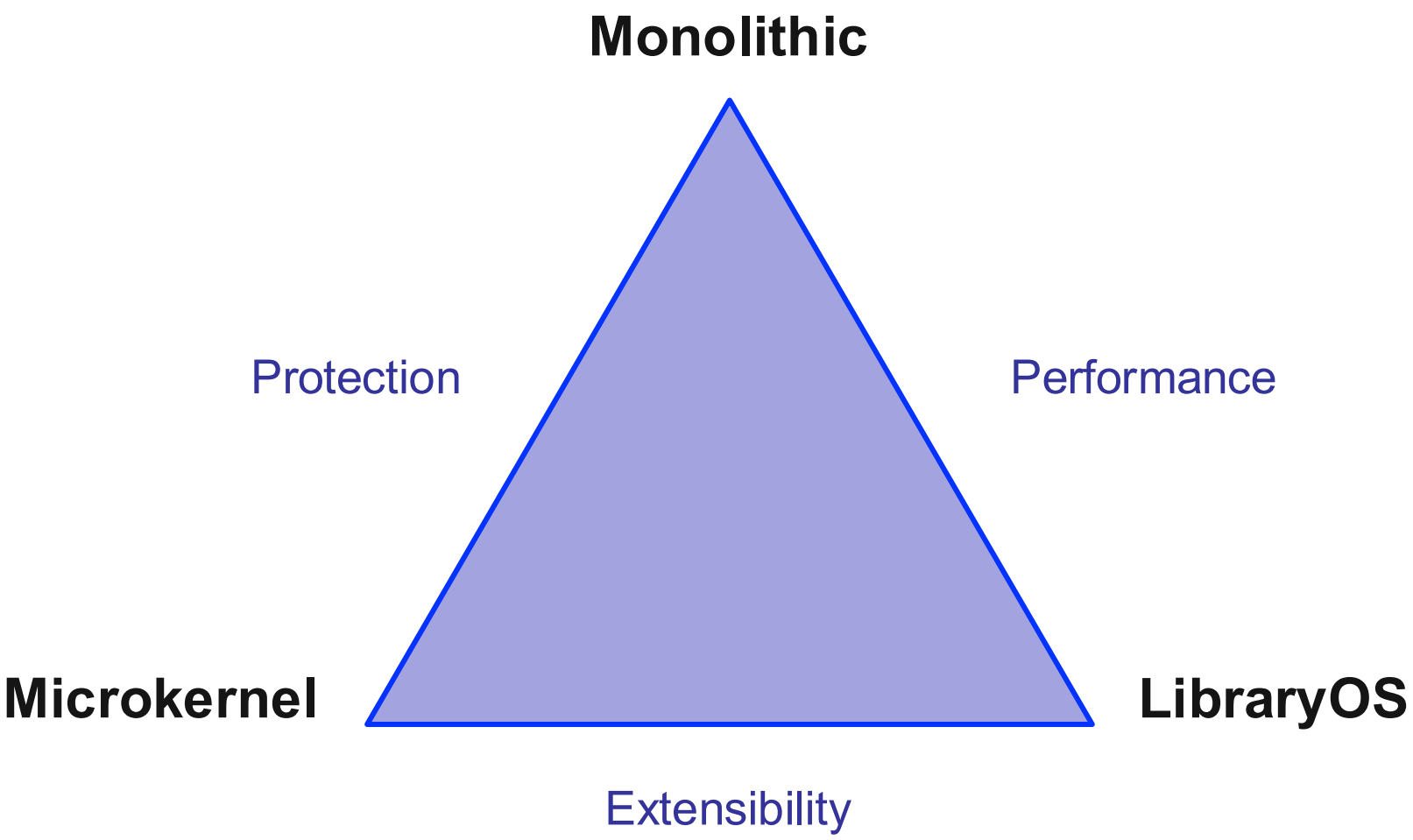
CS 202
Advanced Operating Systems

Winter 26

Lecture 5: Safe Extension of Kernel

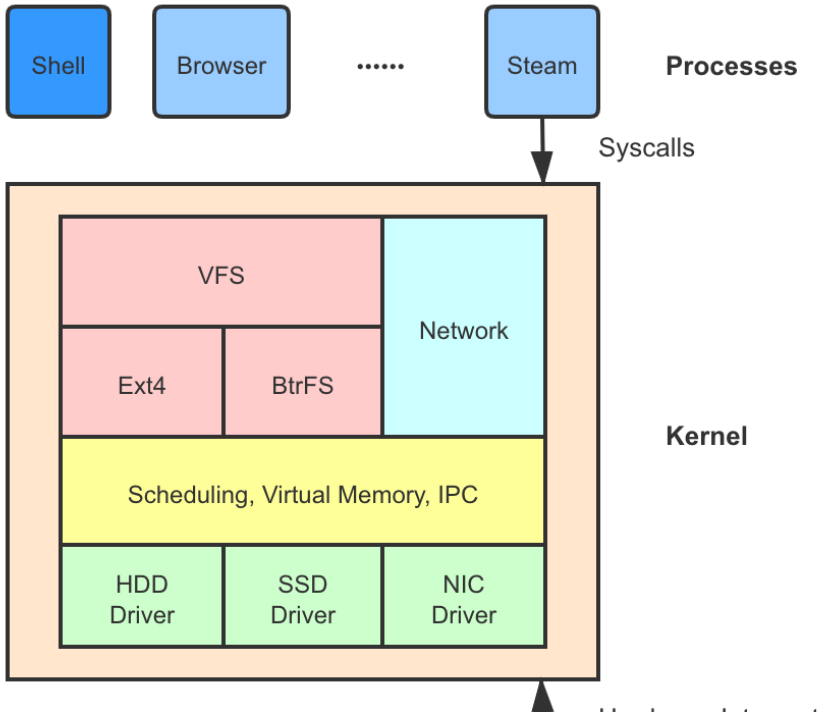
Instructor: Chengyu Song

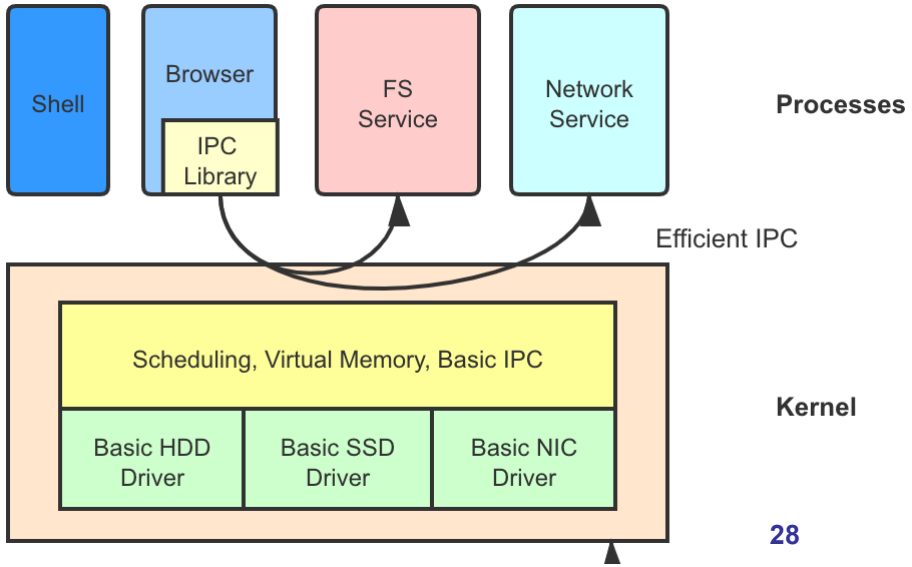
Trade-offs of OS architectures



SFI: Software Fault Isolation

- Isolation mechanisms
  - Hardware
    - » Virtual address space (page, segment)
    - » Fast check but context switch is expensive
  - Software
    - » Language-based (SPIN)
    - » **Sandboxing (SFI)**
      - **SFI ≈ memory protection in software**
  - Hybrid (software + hardware)



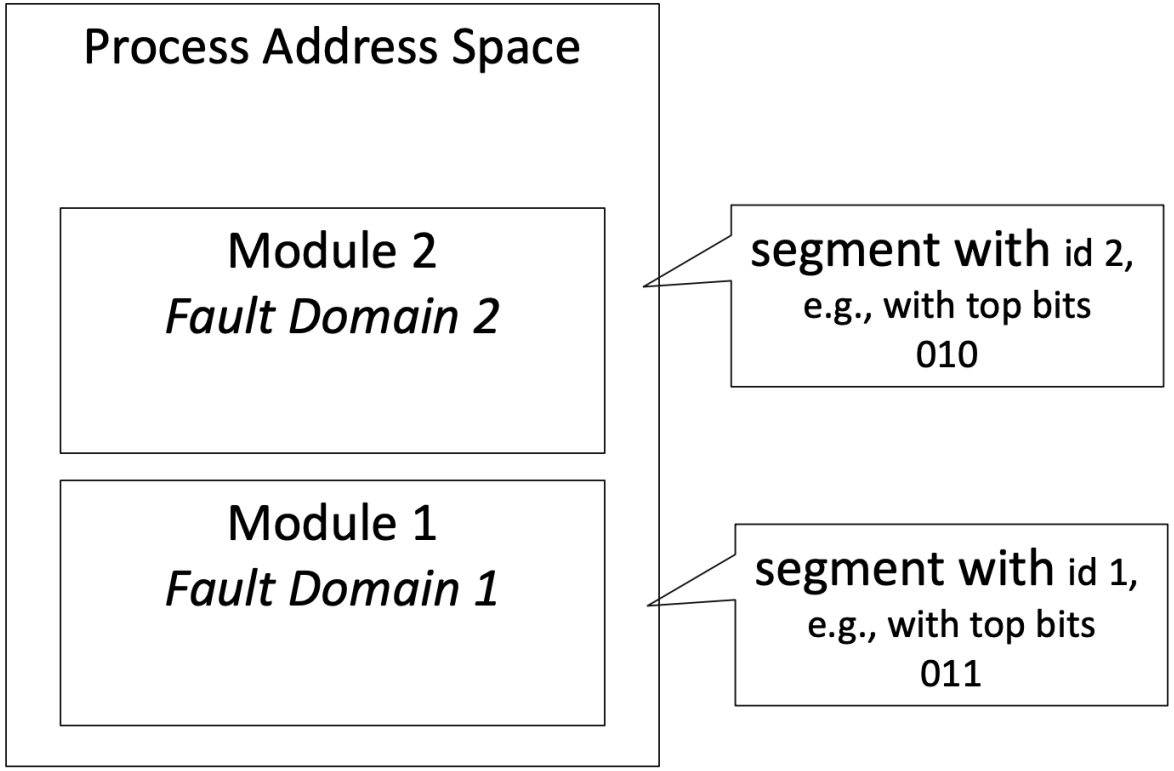


SFI: Goals

- Run untrusted code (plugins, drivers, 3rd-party modules) in the same address space as trusted code with near-native speed
- Confine faults inside distrusted extensions
  - codec shouldn't compromise media player
  - device driver shouldn't compromise kernel
  - plugin shouldn't compromise web browser
- Allow for efficient cross-domain calls
  - numerous calls between media player and codec
  - numerous calls between device driver and kernel

SFI: Core idea

- Logical Fault Domains (Sandboxes) in one address space
  - Divide the single virtual address space into fault domains (one per untrusted module).
  - Each domain gets its own dedicated code region + data region.
  - Trusted code can call into untrusted code; untrusted code cannot escape its sandbox.
  - Key insight: Safety enforced by software checks, not MMU rings.

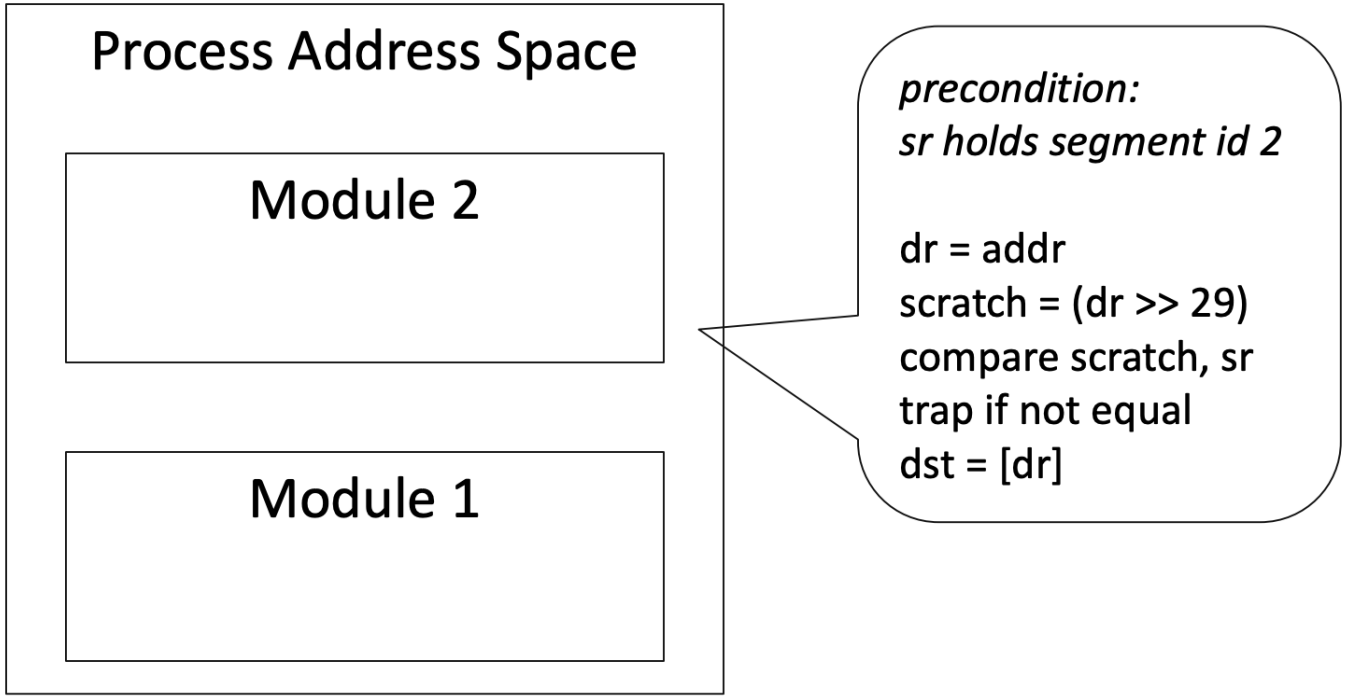


SFI: Inline reference monitor

- What does hardware reference monitor (memory protection) checks?
  - Memory access:
    - » Read/load
    - » Write/store
    - » Execute (control transfer)
- How SFI works?
  - Confine memory access (policies)
    - » Make sure load/store can only access an extension's own fault domain
    - » Make sure control transfer is not arbitrary
  - Mechanism
    - » Compiler-based instrumentation
    - » Binary rewriting / instrumentation

SFI: memory access

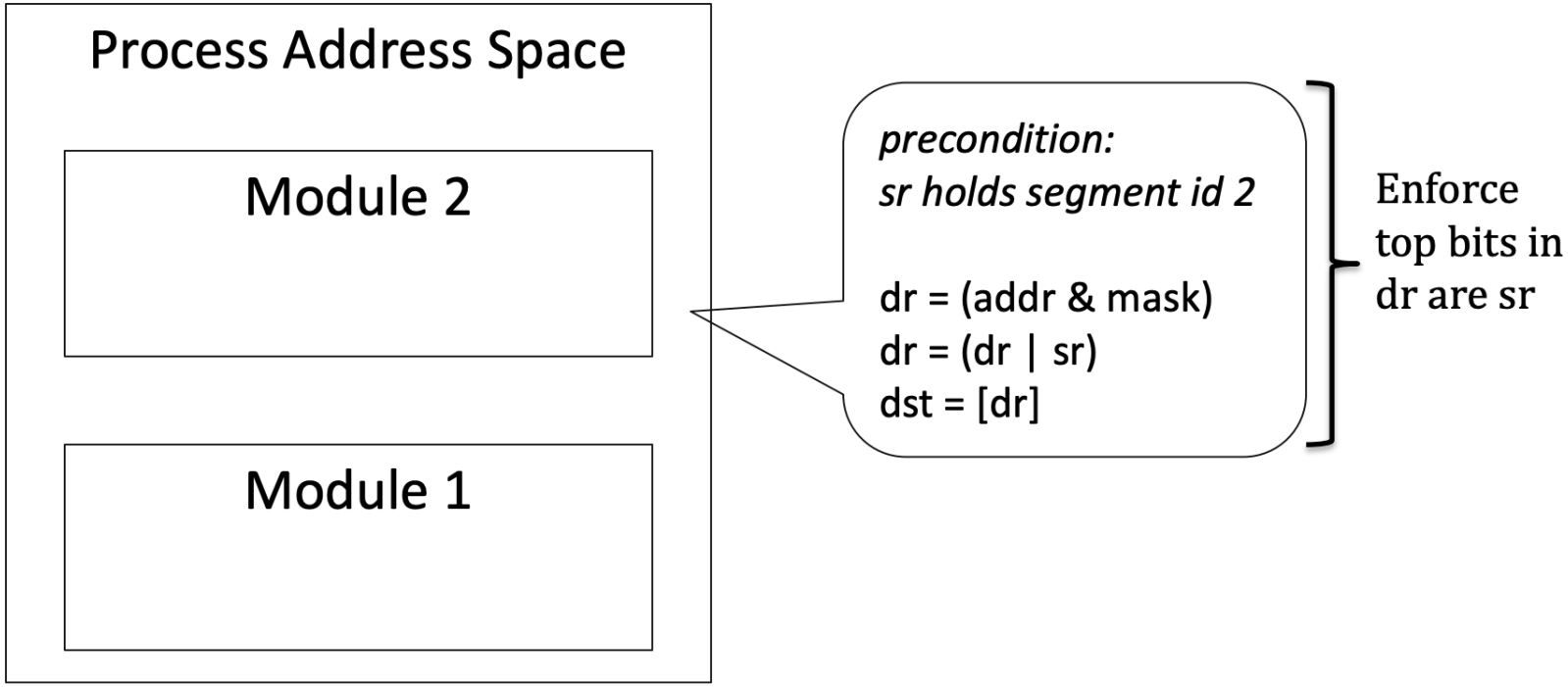
- Scheme 1: (software) Segment Matching
  - Check every mem access for matching seg id
  - Assume dedicated registers segment register (sr) and data register (dr)
    - » not available to the program (no big deal in Alpha)



Safety Requirements:
- **Segment matching code must always be run to ensure safety**
- **Dedicated registers must not be writeable by module**

SFI: memory access

- Scheme 2: Sandboxing
  - Force top bits to match seg id and continue
  - No comparison is made



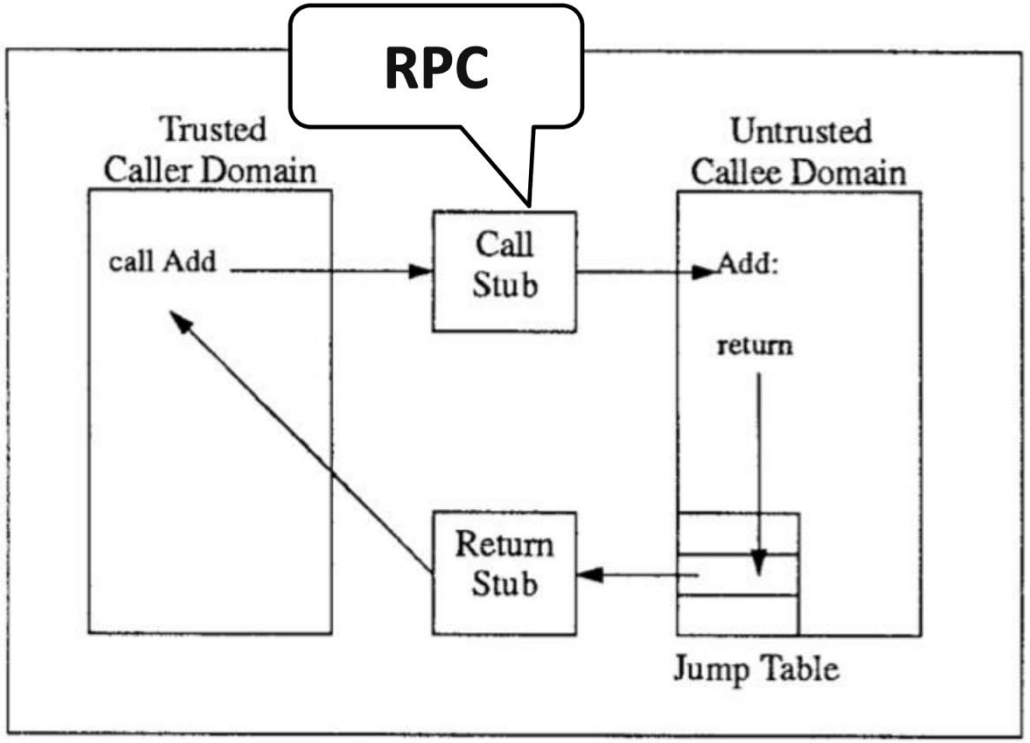
SFI: memory access

- Segment matching
  - More instructions
  - Can pinpoint exact point of fault where segment id doesn’t match

- Sandboxing
  - Fewer instructions
  - Just ensures memory access stays in region (crash is ok)

SFI: control-transfer

- Control-flow safety / integrity is more challenging
  - **Why?**
- Enforcing expected control-flow transfer
  - **Align code blocks**
  - Jumps must land on valid entry points
  - No indirect jumps/calls outside sandbox
- Cross-domain calls
  - Only stubs allowed to make cross-domain jumps
  - Jump table contains allowed exit points
    - » Addresses are hard coded, read-only segment



SFI: data sharing

- All data is readable from fault domains
- Pages mapped into multiple fault domains allow cross-fault-domain communication
- Cross-domain call stubs
  - Copying arguments
  - Preserving machine state
  - Trapping failures and time-outs

SFI: performance

- Microbenchmarks

Benchmark		DEC-MIPS					DEC-ALPHA	
		Fault Isolation Overhead	Protection Overhead	Reserved Register Overhead	Instruction Count Overhead	Fault Isolation Overhead (predicted)	Fault Isolation Overhead	Protection Overhead
052.alvinn	FP	1.4%	33.4%	-0.3%	19.4%	0.2%	8.1%	35.5%
bps	FP	5.6%	15.5%	-0.1%	8.9%	5.7%	4.7%	20.3%
cholesky	FP	0.0%	22.7%	0.5%	6.5%	-1.5%	0.0%	9.3%
026.compress	INT	3.3%	13.3%	0.0%	10.9%	4.4%	-4.3%	0.0%
056.ear	FP	-1.2%	19.1%	0.2%	12.4%	2.2%	3.7%	18.3%
023.eqntott	INT	2.9%	34.4%	1.0%	2.7%	2.2%	2.3%	17.4%
008.espresso	INT	12.4%	27.0%	-1.6%	11.8%	10.5%	13.3%	33.6%
001.gcc1.35	INT	3.1%	18.7%	-9.4%	17.0%	8.9%	NA	NA
022.li	INT	5.1%	23.4%	0.3%	14.9%	11.4%	5.4%	16.2%
locus	INT	8.7%	30.4%	4.3%	10.3%	8.6%	4.3%	8.7%
mp3d	FP	10.7%	10.7%	0.0%	13.3%	8.7%	0.0%	6.7%
psgrind	INT	10.4%	19.5%	1.3%	12.1%	9.9%	8.0%	36.0%
qcd	FP	0.5%	27.0%	2.0%	8.8%	1.2%	-0.8%	12.1%
072.sc	INT	5.6%	11.2%	7.0%	8.0%	3.8%	NA	NA
tracker	INT	-0.8%	10.5%	0.4%	3.9%	2.1%	10.9%	19.9%
water	FP	0.7%	7.4%	0.3%	6.7%	1.5%	4.3%	12.3%
Average		**4.3%**	**21.8%**	**0.4%**	**10.5%**	**5.0%**	**4.3%**	**17.6%**

Table 1: Sandboxing overheads for DEC-MIPS and DEC-ALPHA platforms. The benchmarks `001.gcc1.35` and `072.sc` are dependent on a pointer size of 32 bits and do not compile on the DEC-ALPHA. The predicted fault isolation overhead for **cholesky** is negative due to conservative interlocking on the MIPS floating-point unit.

SFI: performance

- Cross fault-domain calls

Platform	Cross Fault-Domain RPC			C Procedure Call	Pipes
	Caller Save Registers	Save Integer Registers	Save Integer+Float Registers		
DEC-MIPS	1.11μs	1.81μs	2.83μs	0.10μs	204.72μs
DEC-ALPHA	0.75μs	1.35μs	1.80μs	0.06μs	227.88μs

Table 2: Cross-fault-domain crossing times.

SFI: performance

- Postgres DB

Sequoia 2000 Query	Untrusted Function Manager Overhead	Software-Enforced Fault Isolation Overhead	Number Cross-Domain Calls	DEC-MIPS-PIPE Overhead (predicted)
Query 6	1.4%	1.7%	60989	18.6%
Query 7	5.0%	1.8%	121986	38.6%
Query 8	9.0%	2.7%	121978	31.2%
Query 10	9.6%	5.7%	1427024	31.9%

Table 3: Fault isolation overhead for POSTGRES running Sequoia 2000 benchmark.

SFI: limitations

- Original SFI best on RISC (alignment easier).
- Later work (XFI, NaCl) adapts to x86/CISC with alignment + validation.
- Not perfect against all side channels or sophisticated attacks (modern mitigations needed).
- Still requires trusted rewriter/compiler.

SFI: modern legacy

- Most (in-process) sandboxing techniques inherit similar design principles
  - Google Native Client (NaCl) – SFI for running native x86 in Chrome.
  - WebAssembly (Wasm) uses SFI-style sandboxing.
  - eBPF verifier, browser plugins, secure enclaves.
  - Foundation for Control-Flow Integrity (CFI) research.

SFI: comparison with SPIN

Aspect	SFI (1993)	SPIN (1995)
Protection	Runtime software checks (masking)	Language safety (Modula-3 types) + link-time
Language	Any (C, binary)	Safe language only
Overhead	1–10% on untrusted code	Near zero (type-checked at compile)
Trust Model	Trusted binary rewriter	Trusted compiler + type system
Extensibility	Arbitrary untrusted code	Trusted but customizable extensions
Performance Goal	Fast cross-domain calls	Zero-overhead co-located extensions
Use Case	Plugins, drivers, browsers	App-specific kernel services

NaCl: SFI on CISC processors

- Native Client: A Sandbox for Portable, Untrusted x86 Native Code
  - S&P 09'
  - Google Inc
    - » Bennet Yee, David Sehr, Gregory Dardyk, J. Bradley Chen, Robert Muth, Tavis Ormandy, Shiki Okasaka, Neha Narula, and Nicholas Fullagar
  - Best paper award

NaCl: sandboxing browser plugins

- Browser?
  - Is browser an operating system?
  - Why Google?
- What happens before?
  - IE ActiveX nightmares
  - Web-based drive-by download attacks
- Chrome + NaCl
  - Two level sandboxes

NaCl: control-flow constraints

C1	Once loaded into the memory, the binary is not writable, enforced by OS-level protection mechanisms during execution.
C2	The binary is statically linked at a start address of zero, with the first byte of text at 64K.
C3	All indirect control transfers use a `nacljmp` pseudo-instruction (defined below).
C4	The binary is padded up to the nearest page with at least one `hlt` instruction (0xf4).
C5	The binary contains no instructions or pseudo-instructions overlapping a 32-byte boundary.
C6	All *valid* instruction addresses are reachable by a fall-through disassembly that starts at the load (base) address.
C7	All direct control transfers target valid instructions.

NaCl: trust but verify

- A static code verifier ensures:
  - Data integrity
    - All memory addresses are within the sandbox
    - Otherwise, a segmentation fault given (%cs, %ds,… are set)
  - Reliable disassembly
    - All possible jump targets are known (mandatory 32byte alignment for all jump instructions)
  - No unsafe instructions
    - Disassembler is reliable
  - Control flow integrity
    - Same reason for reliable disassembly

NaCl: demo

PCC: Safe Kernel Extensions Without Run-Time Checking

- Untrusted code everywhere: kernel extensions, browser plugins, applets, packet filters, libraries.
- Traditional solutions all have fatal flaws:
  - Trust the source? (Signatures don't prove properties.)
  - Hardware isolation? (Too slow for fine-grained sharing.)
  - Runtime checks? (Overhead kills performance.)
  - Safe language only? (Limits expressiveness.)
- PCC claim: Ship code + formal proof of safety → consumer just verifies the proof.

PCC: core idea

- Code Carries Its Own Safety Certificate
  - Producer (untrusted): writes/compiles code + generates a machine-checkable proof that it obeys the consumer's safety policy.
  - Consumer (trusted): runs a tiny, fast proof checker once before execution.
  - After verification → zero runtime overhead, native speed.
  - Proof is in a logical framework (LF); checking = type-checking the proof.

PCC: safety policies

- What Can Be Proved
  - Policy = set of rules about memory accesses, control flow, resource bounds, calling conventions, etc.
  - Example policy (paper's packet filter): "no out-of-bounds memory access, respects interface pre/postconditions."
  - Proves safety (nothing bad ever happens), not liveness/termination.
  - VCgen (Verification Condition Generator) produces logical obligations for every instruction.

PCC: building proof

- Proofs encoded as terms in the Logical Framework (LF).
- Curry-Howard: “propositions as types, proofs as programs” → checking proof = type-checking.
- Implicit LF optimizations keep proofs compact.
- Certifying compiler (later work) automates proof generation from type-safe source.

PCC: performance

- Proof size: ~few hundred bytes per function (fixed overhead ~250 B).
- Verification time: 1.9 ms on 175 MHz Alpha (~microseconds today).
- End-to-end: Packet filters 10× faster than runtime-checked versions; no overhead after load.
- Trusted computing base (TCB) at consumer: just the proof checker (~few KB).

PCC: limitations

- Proof generation can be hard (needs certifying compiler or theorem prover).
- Proofs can be large for complex policies.
- Not great for highly dynamic code.
- Later work: Typed Assembly Language (TAL), Proof-Transforming Compilation, Foundational PCC, influence on modern formal methods.

PCC: legacy

- Direct ancestors: eBPF verifier, WebAssembly validation, certifying compilers, Rust's borrow checker + formal verification.
- Key insight: "Make the producer do the hard work; consumer only checks."
- PCC shows PL techniques can solve core systems/security problems with minimal trusted code.